notabypass-project

Dokumen ini adalah panduan tunggal untuk membangun sistem "Bypass AI" yang mengubah foto nota menjadi data terstruktur (JSON) tanpa menggunakan logika pemrosesan teks tradisional yang rumit.

1. Product Requirement Document (PRD)

Nama Proyek: notabypass-project
Tujuan Utama: Membantu pemilik bisnis (khususnya UMKM/Konveksi) mencatat pengeluaran secara otomatis hanya dengan memotret nota/struk belanja.
Fitur Utama:

1. **Input Multi-modal:** User bisa mengunggah file gambar (JPG/PNG) atau langsung mengambil foto menggunakan kamera HP via browser.
2. **Local OCR Processing:** Mengubah gambar menjadi teks menggunakan mesin di dalam Docker agar data tidak keluar ke server luar sebelum dibersihkan.
3. **AI Semantic Bypass:** Menggunakan LLM (Gemma-2 via OpenRouter) untuk "membersihkan" teks mentah yang berantakan hasil OCR menjadi format JSON yang siap simpan.
4. **Instant Preview:** Menampilkan hasil olahan AI langsung di layar dalam format yang mudah dibaca.

2. Struktur Direktori (Konfigurasi Folder)

Ini adalah susunan folder yang harus kamu buat di laptopmu. Pastikan semua nama folder menggunakan huruf kecil.

```
notabypass-project/           # Root Proyek
│
├── docker-compose.yml        # Otak operasional Docker
├── data/                     # Jembatan file (Volume) antara Laptop & Docker
├── ocr-worker/               # Layanan Mesin OCR (Python)
│   ├── Dockerfile            # Resep instalasi lingkungan Python
│   ├── requirements.txt      # Daftar belanja library (PaddleOCR, FastAPI)
│   └── main.py               # Kode jembatan API untuk proses OCR
│
└── web/                      # Layanan Frontend (Next.js)
    ├── app/                  # Folder halaman & logika routing
    ├── public/               # File aset (logo, icon)
    └── .env                  # Kunci rahasia (OpenRouter API Key)
```

3. Spesifikasi Teknis

Teknologi yang Digunakan:

- **Frontend:** Next.js (App Router) + Tailwind CSS + Lucide React (Icons).
- **Backend OCR:** Python 3.9 + FastAPI + PaddleOCR (Lebih akurat dari Tesseract).
- **Infrastruktur:** Docker & Docker Compose.
- **AI Gateway:** OpenRouter API (Model: google/gemma-3n-e2b-it:free).

Skema Komunikasi:

1. **User -> Web:** Unggah gambar melalui browser.
2. **Web -> OCR Worker:** Next.js mengirimkan file gambar ke kontainer Docker via HTTP POST.
3. **OCR Worker -> Web:** Mengembalikan teks mentah hasil deteksi gambar.
4. **Web -> OpenRouter:** Next.js mengirim teks mentah tersebut untuk diperbaiki oleh AI.
5. **OpenRouter -> Web:** Mengembalikan data dalam format JSON bersih.

4. Algoritma Utama (Logika Sistem)

Bayangkan ini sebagai aliran air dari hulu (Gambar) ke hilir (Data JSON).

Langkah 1: Tahap Penangkapan (Capture)

- Sistem membuka akses kamera atau galeri.
- Gambar dikonversi menjadi format `multipart/form-data` agar bisa dikirim lewat jaringan.

Langkah 2: Tahap Ekstraksi (The Muscle)

- Gambar masuk ke mesin OCR.
- Mesin melakukan *scanning* baris demi baris.
- Mengabaikan posisi teks (tidak peduli teks miring atau berantakan).
- Output: Satu blok teks mentah panjang.

Langkah 3: Tahap Transformasi (The Brain - The Bypass)

- Teks mentah dikirim ke AI dengan perintah khusus (*System Prompt*).
- **Logika AI:** "Abaikan teks sampah seperti nomor mesin atau ucapan terima kasih. Cari angka yang berdekatan dengan kata 'Total' atau 'Harga'. Ubah ke format JSON."

Langkah 4: Tahap Penyajian (Delivery)

- JSON dikirim kembali ke UI.

- UI melakukan *parsing* dan menampilkan data dalam tabel atau blok kode yang rapi.

5. ADHD-Friendly Guide (Daftar Urutan Kerja)

Jangan loncat! Kerjakan satu kotak per satu sesi fokus.

Fase 1: Persiapan Skeleton (Satu Kali Setup)

- [ ] Buat folder root `notabypass-project`.
- [ ] Buat folder `data`, `ocr-worker`, dan `web`.
- [ ] Buat file `docker-compose.yml` di folder root (sebagai jembatan).

Fase 2: Membangun Otot (OCR Worker)

- [ ] Buat `requirements.txt` di folder `ocr-worker`.
- [ ] Buat `Dockerfile` di folder `ocr-worker`.
- [ ] Buat `main.py` (FastAPI) untuk menerima kiriman gambar.
- [ ] **Uji Coba:** Jalankan `docker-compose up --build` dan pastikan API di port 8000 merespon.

Fase 3: Membangun Wajah (Next.js Frontend)

- [ ] Inisialisasi Next.js di folder `web`.
- [ ] Buat UI Landing Page sederhana dengan tombol upload.
- [ ] Buat fungsi `fetch` untuk mengirim file dari browser ke port 8000 (OCR Docker).

Fase 4: Integrasi Bypass (OpenRouter)

- [ ] Ambil API Key dari OpenRouter.
- [ ] Buat API Route di Next.js untuk meneruskan teks hasil OCR ke OpenRouter menggunakan perintah `fetch` (seperti cURL yang kamu punya).
- [ ] **Final Test:** Masukkan foto nota, dan lihat JSON muncul di layar.

Tips Tambahan:

- Jika kamu merasa bingung, lihat kembali **Struktur Direktori** di atas untuk memastikan posisi filemu benar.
- Fokuslah untuk membuat satu jalur berhasil dulu (dari Upload sampai Teks muncul), baru pikirkan cara merapikan tampilannya.